

React 프로젝트 구조 비교: 폴더 구조부터 Next.js까지

React와 Vite 그리고 React Router와 Axios 그리고 Context와 Tailwind를 여러 프로젝트에서 사용했지만 프로젝트마다 구조는 달랐다. 이 글에서는 CUTY와 SoWhat과 이룸매이트와 GAON [web/](#)을 비교하며 그 이유를 정리한다.

- CUTY: 시작 단계의 전공 프로젝트
- SoWhat: 라우팅과 인증과 사용자 상태와 SSE가 포함된 프로젝트
- 이룸매이트: [app](#)과 [pages](#)와 [features](#)와 [shared](#) 구조
- GAON [web/](#): React + Vite 기반 웹 프로젝트

1. 프로젝트가 커지면 폴더도 나뉜다

처음에는 [components](#)와 [pages](#)와 [api](#) 정도로도 충분하다. 하지만 화면과 상태와 서버 통신과 실시간 연결이 늘어나면 한 파일에 모든 역할을 담기 어려워진다.

폴더가 많아지는 이유는 파일을 많이 만들기 위해서가 아니라 서로 다른 역할을 분리하기 위해서다.

2. CUTY에서 브라우저와 React가 연결되는 과정

index.html -> main.tsx -> createRoot() -> App -> Page -> Component

[index.html](#)에는 React가 들어갈 `#root`만 있고 `createRoot`가 그 자리에 `App`을 렌더링한다.

이처럼 JavaScript가 브라우저에서 화면을 만드는 방식을 CSR(Client-Side Rendering)이라고 한다. Vite는 개발 서버와 빌드를 담당하고 React는 컴포넌트와 상태를 이용해 화면을 만든다.

3. 화면이 늘어나면 Router가 필요하다

화면이 하나일 때는 `App`에서 바로 페이지를 보여줘도 된다. 로그인과 회원가입과 프로필처럼 화면이 늘어나면 React Router가 URL과 화면을 연결한다.

```
BrowserRouter
`-- App
  `-- Routes
    +-- /
    +-- /login
    +-- /profile
    `-- /articles/:id
```

[Link](#)나 이동 함수를 사용하면 앱 전체를 새로 불러오지 않고 URL과 화면을 바꿀 수 있다. 따라서 SPA에서 URL은 화면을 결정하는 중요한 값이다.

4. 기능이 늘어나면 상태와 역할을 분리해야 한다

SoWhat의 `App`에는 라우팅과 헤더와 푸터와 로그인 여부와 사용자 정보와 세션 확인 등이 함께 들어 있었다. 작은 프로젝트에서는 한곳에서 확인하기 편하지만 기능이 늘어나면 서로 다른 역할이 한 파일에 쌓인다.

상태도 사용 범위에 따라 나누는 것이 좋다.

- 모달 열림 여부: 해당 컴포넌트의 상태
- 입력값: 해당 폼이나 기능의 상태
- 로그인 사용자와 세션: 여러 화면이 공유하는 상태

여러 화면에서 함께 사용하는 값은 Context처럼 공통으로 접근할 수 있는 위치에 둘 수 있다. 모든 상태를 전역으로 만들기보다 필요한 범위를 기준으로 선택해야 한다.

5. 이름매이트는 앱과 기능을 나누었다

```
app/
pages/
features/
shared/
```

기능 내부도 필요한 역할에 따라 나뉜다.

```
features/auth/
+-- api/
+-- components/
+-- hooks/
`-- model/
```

이 구조를 완성된 FSD라고 보기보다 앱 전체에서 사용하는 코드와 특정 기능의 코드를 구분한 구조로 이해하는 편이 적절하다.

- **app**: 앱 전체 조립
- **pages**: 화면
- **features**: 개별 기능
- **shared**: 여러 곳에서 재사용하는 코드

인증이나 소켓처럼 페이지가 바뀌어도 유지되어야 하는 값은 Provider로 분리하고 채팅 입력값이나 모달 상태처럼 범위가 좁은 값은 해당 기능 안에 둔다.

폴더 구조는 파일을 예쁘게 정리하는 일이 아니라 코드가 필요한 범위를 나누는 일이다.

6. 버튼을 누른 뒤 데이터는 API를 거쳐 이동한다

사용자 입력 -> Page/Component -> API 함수 -> 공통 HTTP client -> Backend

프로젝트별 흐름은 다음과 같았다.

```
CUTY   Component -> api -> Axios client -> Backend
SoWhat Page -> api -> fetch -> Backend
이름매이트 Page/Component -> feature hook -> feature api -> shared client -> Backend
```

API 요청을 컴포넌트에 직접 작성하면 인증과 에러 처리와 서버 주소 같은 코드가 반복된다. 그래서 API 함수와 공통 HTTP client를 분리하고 반복되는 규칙을 한곳에서 관리한다.

SoWhat의 인증 흐름은 다음과 같다.

일반 요청 -> 401 응답 -> refresh 요청 -> 인증 갱신 -> 원래 요청 재시도

이처럼 Axios interceptor와 cookie 기반 인증과 공통 client는 서로 다른 구현처럼 보여도 반복되는 통신 규칙을 중앙에서 처리한다는 공통점이 있다.

7. 화면 상태와 서버 데이터는 다르다

`useState`에 담긴다고 해서 모든 상태가 같은 것은 아니다.

- 화면 상태: 모달과 드롭다운과 입력값
- 서버 데이터: 게시글과 채팅방과 사용자 정보

서버 데이터는 로딩과 에러와 재요청과 캐시까지 함께 관리해야 한다. 따라서 화면에서만 쓰는 상태와 서버에서 받아온 데이터를 구분해 보는 것이 중요하다. TanStack Query 같은 도구는 이런 서버 상태를 관리하기 위해 사용된다.

8. 폴더 구조는 코드를 찾는 지도다

초기에는 기술별 구조가 직관적이다.

```
components/  
hooks/  
api/  
types/
```

하지만 채팅처럼 하나의 기능에 여러 종류의 코드가 필요해지면 관련 파일이 흩어진다. 이때는 기능별 구조가 더 편할 수 있다.

```
features/  
+-- auth/  
`-- chat/  
    +-- api/  
    +-- components/  
    +-- hooks/  
    `-- model/
```

작은 프로젝트에서 지나치게 세분화하면 오히려 복잡해지고 큰 프로젝트에서 모든 코드를 한곳에 두면 수정 범위를 파악하기 어렵다. 중요한 것은 FSD나 특정 구조를 그대로 따르는 일이 아니라 팀이 코드를 쉽게 찾고 수정할 수 있는 기준을 정하는 일이다.

9. 스타일링 도구는 달라도 해결하는 문제는 같다

- CUTY: Tailwind CSS + Global CSS
- SoWhat: CSS Module
- 이룸매이트: Tailwind CSS + Shared UI
- GAON [web/](#): Tailwind CSS + Radix UI + MUI + Emotion

Global CSS는 공통 스타일을 관리하기 쉽지만 범위가 넓다. CSS Module은 컴포넌트별 스타일을 분리하기 좋고 Tailwind CSS는 클래스를 조합해 빠르게 작성할 수 있다. Radix UI나 MUI는 버튼과 모달 같은 UI 구현을 줄여준다.

여러 도구를 함께 사용하수록 어떤 상황에 어떤 도구를 쓸지 기준이 필요하다. CSS는 모든 속성을 외우기보다 다음 기본 원리를 먼저 이해하는 편이 좋다.

box model -> position -> flex -> grid -> responsive -> overflow

도구는 바뀌어도 브라우저가 화면을 배치하는 기본 원리는 크게 달라지지 않는다.

10. React에서 Next.js로 이어지는 이유

React와 Vite 프로젝트는 주로 브라우저에서 JavaScript가 화면을 만드는 CSR에 가깝다. 하지만 HTML을 만드는 시점은 여러 가지가 있다.

CSR: 요청 -> JavaScript 전달 -> 브라우저에서 화면 생성
SSR: 요청 -> 서버에서 HTML 생성 -> 브라우저에 전달
SSG: 빌드 시 HTML 생성 -> 요청 시 미리 만든 HTML 전달

Next.js는 React를 기반으로 CSR과 SSR과 SSG를 함께 다룰 수 있고 서버 기능도 한 프로젝트에서 구성할 수 있다. 따라서 React 다음에 Next.js를 공부한다는 것은 단순히 새로운 문법을 배우는 것이 아니라 화면을 언제 어디에서 만들 것인지까지 고민하는 단계로 넘어가는 일이다.

마무리

프로젝트를 비교하면서 기술은 따로 떨어진 목록이 아니라 하나의 흐름으로 연결되어 있다는 점을 확인할 수 있었다.

Browser -> React App -> Router -> Page -> Feature/Component -> State -> API -> Backend

프로젝트마다 구조가 다른 이유는 필요한 화면과 기능과 팀의 작업 방식이 다르기 때문이다. 따라서 구조를 볼 때는 어떤 폴더 구조가 정답인지보다 각 코드가 어떤 역할을 하고 어느 범위에서 사용되는지를 먼저 보는 편이 좋다.

좋은 구조는 하나로 정해져 있는 것이 아니라 프로젝트의 상황에 맞게 선택하는 구조다.

참고한 프로젝트

- [CUTY client](#)
- [SoWhat client](#)
- [이룸매이트 client](#)
- [GAON](#)